Problem Set 2

Due date: March 18, 2026

倪伟丰 2024110089

1. (Exercise 2.3 from the textbook) Take the following list of functions and arrange them in ascending order of growth rate. That is, if function $g(n)$ immediately follows function $f(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$$f_1(n) = n^{2.5}$$
$$f_2(n) = \sqrt{2n}$$
$$f_3(n) = n + 10$$
$$f_4(n) = 10^n$$
$$f_5(n) = 100^n$$
$$f_6(n) = n^2 \log n$$

解：

递增顺序为：

$$f_2 \prec f_3 \prec f_6 \prec f_1 \prec f_4 \prec f_5$$

逐步说明如下：

- $f_2 = O(f_3)$：$f_2(n) = \sqrt{2n} = \Theta(\sqrt{n})$，而 $\sqrt{n} = O(n) = O(n+10)$，故 $f_2 = O(f_3)$。
- $f_3 = O(f_6)$：$f_3(n) = n+10 = \Theta(n)$，而 $n = O(n^2) = O(n^2 \log n)$，故 $f_3 = O(f_6)$。
- $f_6 = O(f_1)$：$\frac{f_6(n)}{f_1(n)} = \frac{n^2 \log n}{n^{2.5}} = \frac{\log n}{n^{0.5}} \to 0$（$n \to \infty$），故存在 n_0 使得当 $n \geq n_0$ 时 $f_6(n) \leq f_1(n)$，即 $f_6 = O(f_1)$。
- $f_1 = O(f_4)$：对任意 $\varepsilon > 0$，当 n 充分大时 $n^{2.5} \leq (1+\varepsilon)^n \leq 10^n$，故 $f_1 = O(f_4)$。
- $f_4 = O(f_5)$：$\frac{f_4(n)}{f_5(n)} = \frac{10^n}{100^n} = \left(\frac{1}{10}\right)^n \to 0$，故存在 n_0 使得当 $n \geq n_0$ 时 $f_4(n) \leq f_5(n)$，即 $f_4 = O(f_5)$。

2. (Exercise 2.5 from the textbook) Assume you have functions f and g such that $f(n)$ is $O(g(n))$. For each of the following statements, decide whether you think it is true or false and give a proof or counterexample.

   (a) $\log_2 f(n)$ is $O(\log_2 g(n))$.

   (b) $2^{f(n)}$ is $O(2^{g(n)})$.

   (c) $f(n)^2$ is $O(g(n)^2)$.

   (a) **错误**。反例：取 $f(n) = 2$，$g(n) = 1$。

   取 $c = 2$，$n_0 = 1$，则对所有 $n \geq n_0$ 均有 $f(n) = 2 \leq 2 \cdot 1 = c \cdot g(n)$，故 $f(n) = O(g(n))$。

   但 $\log_2 f(n) = \log_2 2 = 1$，而 $\log_2 g(n) = \log_2 1 = 0$。

   由于 $\lim_{n\to\infty} \frac{\log_2 f(n)}{\log_2 g(n)} = \infty$，故 $\log_2 f(n)$ 不是 $O(\log_2 g(n))$。

   (b) **错误**。反例：取 $f(n) = 2n$，$g(n) = n$。

   取 $c = 2$，$n_0 = 1$，则对所有 $n \geq n_0$ 均有 $f(n) = 2n \leq 2n = c \cdot g(n)$，故 $f(n) = O(g(n))$。

   但 $\frac{2^{f(n)}}{2^{g(n)}} = \frac{2^{2n}}{2^n} = 2^n \to \infty$（$n \to \infty$）。

   因此不存在常数 $c' > 0$ 使得对所有充分大的 n 均有 $2^{f(n)} \leq c' \cdot 2^{g(n)}$，故 $2^{f(n)}$ 不是 $O(2^{g(n)})$。

   (c) **正确**。因为 $f(n) = O(g(n))$，存在常数 $c > 0$ 和 $n_0 \geq 0$，使得对所有 $n \geq n_0$ 均有

   $$f(n) \leq c \cdot g(n)$$

   两边平方得

   $$f(n)^2 \leq c^2 \cdot g(n)^2$$

   取常数 $c' = c^2$，则对所有 $n \geq n_0$ 均有 $f(n)^2 \leq c' \cdot g(n)^2$，故 $f(n)^2 = O(g(n)^2)$。

3. (Exercise 2.6 from the textbook) Consider the following basic problem. You're given an array A consisting of n integers $A[1], A[2], \ldots, A[n]$. You'd like to output a two-dimensional n-by-n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$—that is, the sum $A[i] + A[i + 1] + \ldots + A[j]$. (The value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$, so it doesn't matter what is output for these values.) Here's a simple algorithm to solve this problem.

for $i = 1, 2, \cdots, n$ **do**
  for $j = i+1, i+2, \cdots, n$ **do**
    Add up array entries $A[i]$ through $A[j]$
    Store the result in $B[i, j]$
  end for
end for

(a) For some function f that you should choose, give a bound of the form $O(f(n))$ on the running time of this algorithm on an input of size n (i.e., a bound on the number of operations performed by the algorithm).

(b) For this same function f, show that the running time of the algorithm on an input of size n is also $\Omega(f(n))$. (This shows an asymptotically tight bound of $\Theta(f(n))$ on the running time.)

(c) Although the algorithm you analyzed in parts (a) and (b) is the most natural way to solve the problem—after all, it just iterates through the relevant entries of the array B, filling in a value for each—it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve this problem, with an asymptotically better running time. In other words, you should design an algorithm with running time $O(g(n))$, where $\lim_{n\to\infty} g(n)/f(n) = 0$.

解：

(a) 取 $f(n) = n^3$，该算法的运行时间为 $O(n^3)$。

对每一对满足 $1 \le i < j \le n$ 的下标，计算 $B[i, j] = A[i] + \cdots + A[j]$ 需要 $j - i$ 次加法，故总操作次数为

$$T(n) = \sum_{i=1}^{n} \sum_{j=i+1}^{n} (j - i).$$

令 $k = j - i$，则

$$T(n) = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} k = \sum_{i=1}^{n-1} \frac{(n-i)(n-i+1)}{2} \le \sum_{i=1}^{n-1} \frac{n^2}{2} = \frac{(n-1)n^2}{2} = O(n^3)$$

(b) 该算法的运行时间也是 $\Omega(n^3)$。

考虑满足 $1 \le i \le \lfloor n/4 \rfloor$ 且 $\lceil 3n/4 \rceil \le j \le n$ 的下标对 (i, j)。这样的对至少有 $\lfloor n/4 \rfloor \cdot \lfloor n/4 \rfloor = \Omega(n^2)$ 个，且每个区间长度满足

$$j - i \ge \left\lceil \frac{3n}{4} \right\rceil - \left\lfloor \frac{n}{4} \right\rfloor \ge \frac{n}{2}.$$

因此

$$T(n) \ge \Omega(n^2) \cdot \frac{n}{2} = \Omega(n^3).$$

结合 (a) 的上界，得 $T(n) = \Theta(n^3)$。

(c) 利用前缀和可以将运行时间优化至 $O(n^2)$。算法如下：

$S[0] \leftarrow 0$
for $k = 1, 2, \cdots, n$ **do**
  $S[k] \leftarrow S[k-1] + A[k]$
end for
for $i = 1, 2, \cdots, n$ **do**
  for $j = i+1, i+2, \cdots, n$ **do**
    $B[i, j] \leftarrow S[j] - S[i-1]$
  end for
end for

正确性：由前缀和定义 $S[k] = A[1] + \cdots + A[k]$，故

$$S[j] - S[i-1] = A[i] + A[i+1] + \cdots + A[j] = B[i, j].$$

时间复杂度：构造前缀和数组 S 需要 $O(n)$ 次操作；双重循环 $= \Theta(n^2)$ 次，每次仅需一次减法（$O(1)$）。故总运行时间为 $O(n^2)$。

由于 $\lim_{n\to\infty} \dfrac{n^2}{n^3} = 0$，该算法渐近复杂度严格优于原算法。